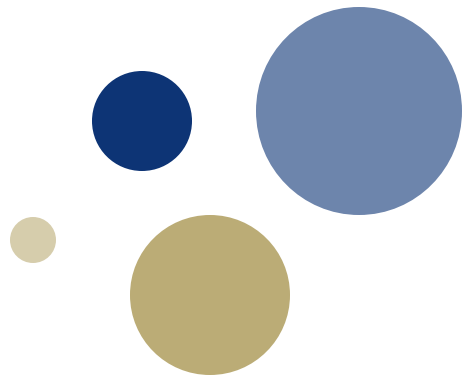




NTNU

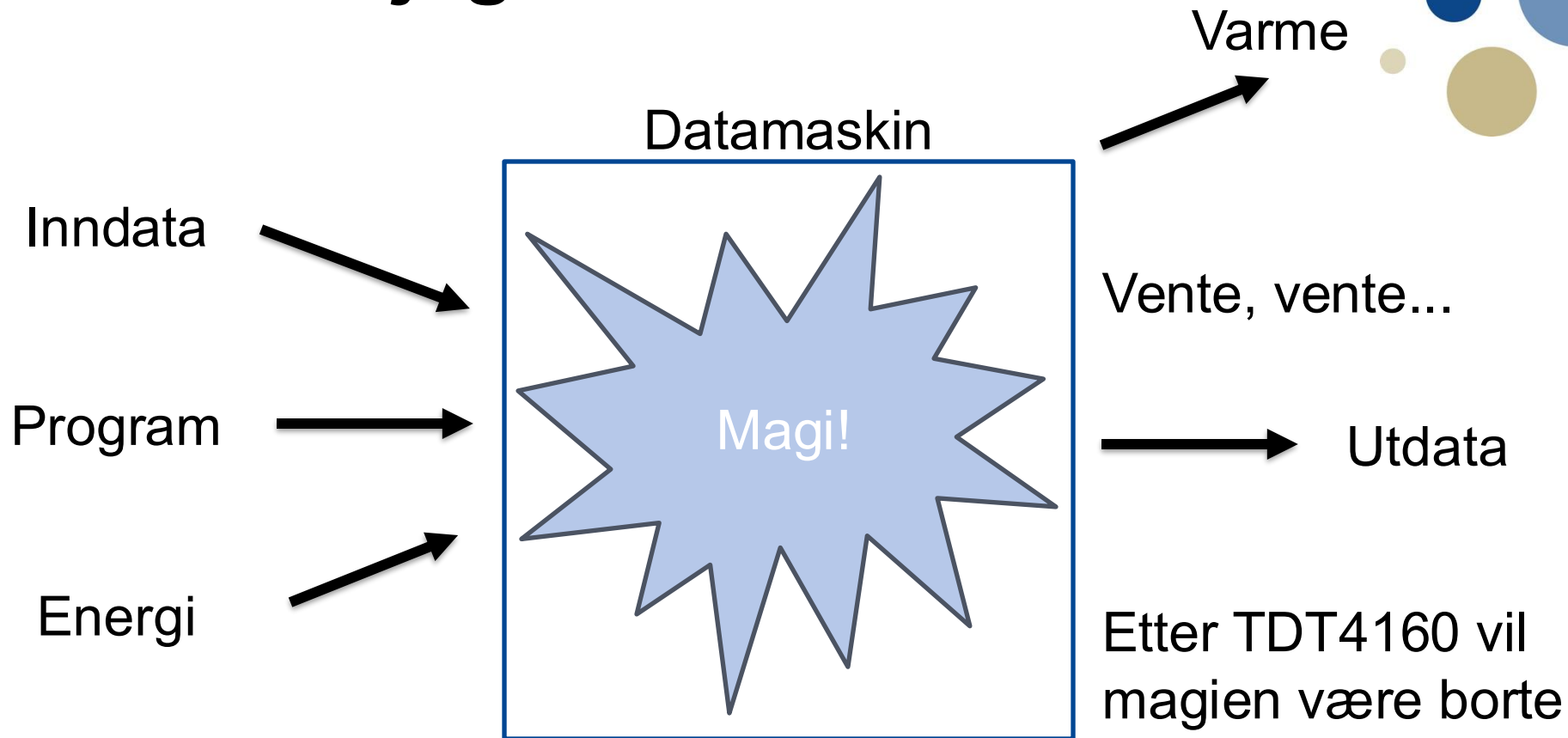
Kunnskap for en bedre verden



Forelesning 1: Introduksjon og ytelse

TDT4160 Datamaskiner

Hva skal jeg lære i TDT4160?



Hva skal jeg lære i TDT4160?

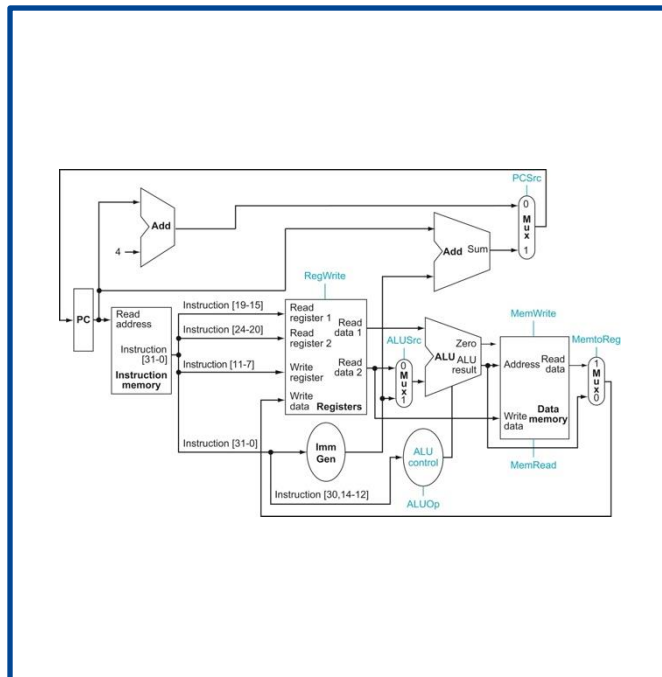
Varme

Datamaskin

Inndata

Program

Energi



Vente, vente...

Utdata

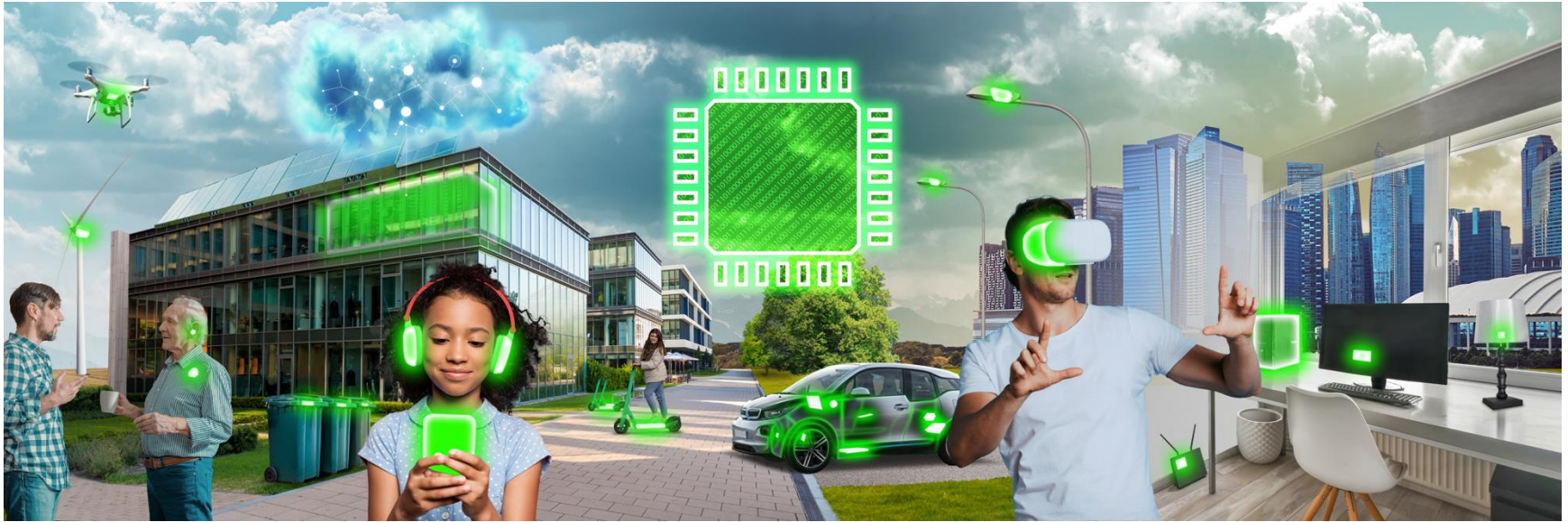
Etter TDT4160 vil magien være borte

Datamaskiner er overalt



[Image by [EECS](#), NTNU]

Datamaskiner er overalt



[Image by [EECS](#), NTNU]

Hva kan dere fra før?

- Alle har brukt datamaskiner i årevis
- Alle bør kunne programmere
 - Det er ikke så farlig hvilket programmeringsspråk du er vant til
 - Vi fokuserer på de fundamentale konstruksjonene (for eksempel beslutninger, løkker, og funksjoner)
- Noen kan en del digitalteknikk, andre kan ingenting
 - Forelesning E1 og E2 dekker den digitalteknikken som er nødvendig for å følge kurset
 - Hvis tok TTT4203 Innføring i analog og digital elektronikk og hang høvelig godt med, er E1 og E2 kun repetisjon

TDT4160 er starten...

Datamaskinsystem

Programvare

TDT4205

Kompilator

Operativsystem

En drøss med emner

TDT4186

Maskinnær programmering

TDT4258

Maskinvare

TDT4160

Makroarkitektur

TDT4260

Mikroarkitektur

TDT4255

Digital elektronikk

En drøss med emner

Analog elektronikk

En drøss med emner



... og for andre er TDT4160 «slutten»

- Applikasjoner som gjør nøyaktig samme beregning kan ha vidt forskjellig ytelse
- Forskjellen kommer av hvor godt applikasjonen utnytter ressursene i datamaskinen
- TDT4160 gir deg en grunnleggende forståelse av hva datamaskinen kan gjøre
 - Fokus på fundamentale prinsipper
 - «Kunnskap med lang halveringstid»

Eksempel: Matrisemultiplikasjon

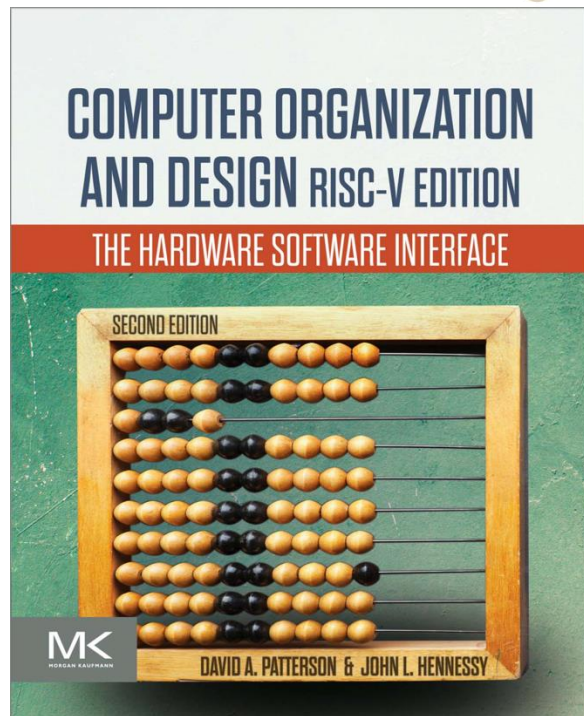
$$C = AB$$

```
for i in range(n):  
    for j in range(n):  
        for k in range(n):  
            C[i][j] +=  
                A[i][k] * B[k][j]
```

<i>n</i>	Naiv	Numpy	Speedup
128	0.223 s	0.004 s	~55X
1024	101.5 s	0.1 s	~1000X

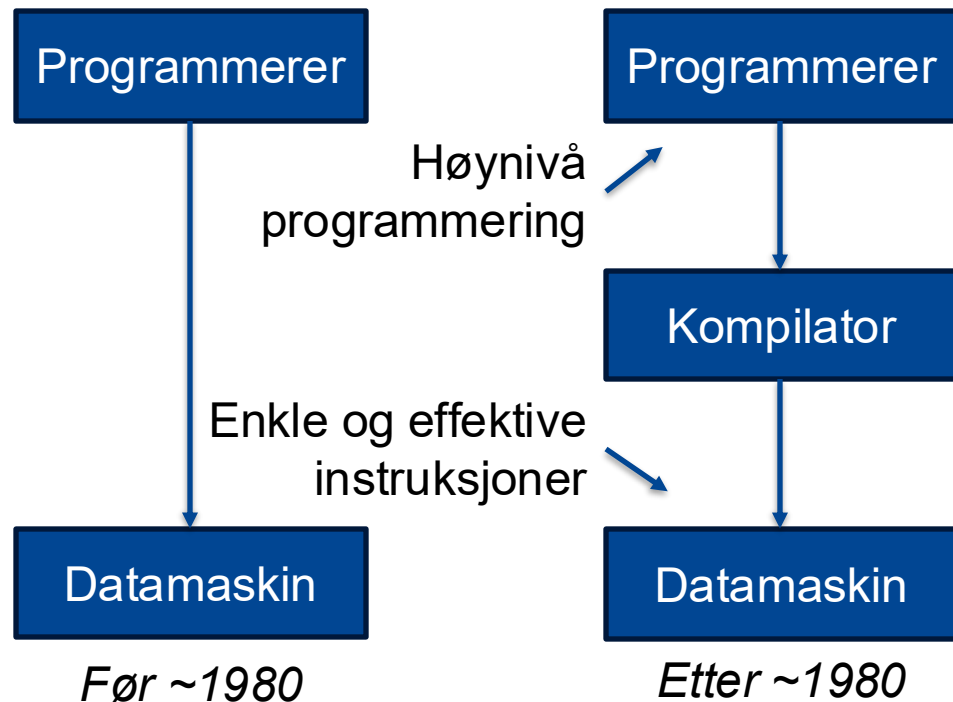
Foreløpig pensum og lærebok

- Følgende kapitler i Patterson and Hennessy er pensum:
 - Kapittel 1.1 til 1.13
 - Kapittel 2.1 til 2.12, 2.14, 2.22 og 2.23
 - Kapittel 3.1 til 3.6, 3.9 og 3.10
 - Kapittel 4.1 til 4.11, 4.15 og 4.16
 - Kapittel 5.1 til 5.4, 5.6, 5.7, 5.10, 5.16 og 5.17
 - Kapittel 6.1 til 6.8, 6.11, 6.14 og 6.15
 - Appendiks A-1 til A-3, A-5, A-7 til A-11 (unntatt beskrivelsene av Verilog-kode)
- I tillegg er øvingene og forelesningene pensum



Hvorfor denne boka?

- Turingprisen er nobelprisen i datateknologi
- Hennesy og Patterson fikk Turingprisen i 2017 blant annet for den kvantitative metoden som beskrives i boka
- Denne metoden er mye av grunnen til at vi bygger datamaskiner på den måten vi gjør i dag



Semesterplan

Praktiske øvinger													
PØ1				PØ2				PØ3				PØ4	
Teoriøvinger													
TØ1		TØ2			TØ3			TØ4				TØ5	
Forelesninger													
Ø1		E1		E2									
F1	F2	F3	F4	F5	F6	F7	F8		F9	F10	F11	F12	F13
Uke 34	Uke 35	Uke 36	Uke 37	Uke 38	Uke 39	Uke 40	Uke 41	Uke 42	Uke 43	Uke 44	Uke 45	Uke 46	Uke 47

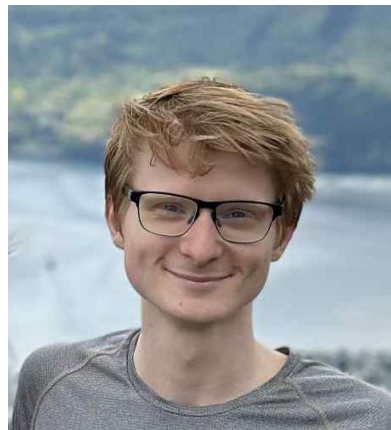
- En forelesning i uka med unntak av uke 35, 36, 38, og 41.
 - Hovedforelesningene (F) holdes på tirsdagene (fordi vi har det store auditoriet R1)
 - Mandagstimen brukes til øvingsforelesninger (Ø1) og digitalteknikkforelesningene (E1 og E2), men da har vi et mindre auditorium (A2)
 - Alle forelesninger blir tatt opp og gjort tilgjengelig på Blackboard
 - Forelesningspausen i uke 41 er på grunn av skolens høstferie
- Øvingsopplegget består av fem teoriøvinger og fire praktiske øvinger
 - Alle øvingene må være godkjent for å få gå opp til eksamen

Øvingsopplegg

- Øvingene leveres og evalueres i INGIInious og starter neste uke:
 - <https://tdt4160.idi.ntnu.no/>
- Teoriøvingene skal hjelpe dere med å forstå hvordan datamaskinen virker (som vil komme godt med på eksamen)
 - Mål: Det skal lønne seg å gjøre øvingene ordentlig
- De praktiske øvingene er assemblyprogrammering i Ripes
 - Bruk <https://ripes.me/> eller installer Ripes på egen maskin
- Det blir en egen forelesning om øvingsopplegget i uke 35 (Ø1)
- Ikke kok – det går kun utover deg selv

Fagstaben

- Faglærer Magnus Jahre
 - Professor i datamaskinarkitektur
 - Forsker på ytelsesanalyse, ytelsesmodellering, akseleratorer, og datasystemer som baserer seg på energihøsting.
- Vit. ass. Håvard Rognebakke Krogstie
 - Stipendiat i kompilorteknikk
 - Forsker på kompilatorer, statisk analyse av minneoperasjoner, korrekthet, og interne representasjoner.
- 11 læringsassistenter



Hvordan få hjelp?

- Blackboard
 - All informasjon om faget distribueres via Blackboard
- Læringsassistentene tilbyr en-til-en veiledning på sal
 - Operativ fra uke 35 (neste uke)
- Vi bruker et forum til spørsmål og svar:
 - <https://tdt4160.idi.ntnu.no/forum/>
 - Logg på med NTNU **brukernavn** og passord
 - Alle spørsmål på forumet skal besvares i løpet av neste virkedag
 - Unngå epost (men bruk epost når du må)
 - Det går an å være anonym, men fagstaben kan se hvem du er

Øvingstimer

LA = Læringsassistent

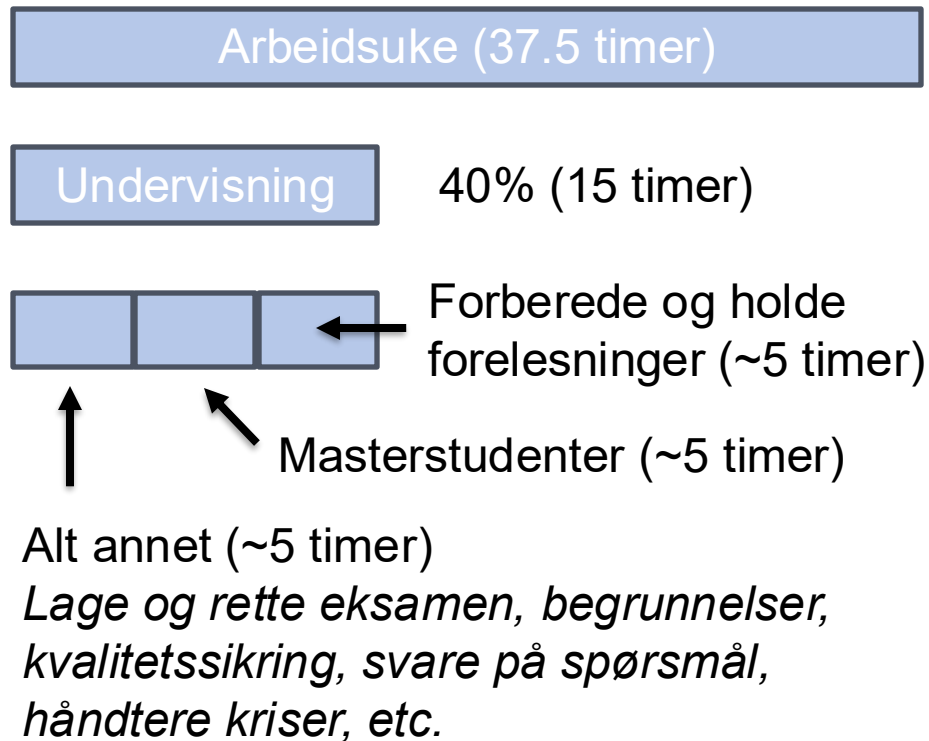
Uke 34	ma. 18.8	ti. 19.8	on. 20.8	to. 21.8	fr. 22.8
07					
08	08:15 - 10:00 2-Laboratorieøvelse • S21 1 LA	08:15 - 10:00 6-Laboratorieøvelse • Lise 1 LA		08:15 - 10:00 1-Laboratorieøvelse • VE20 2 LAer	08:15 - 10:00 7-Laboratorieøvelse • VE22 - SMASH 2 LAer
09					
10	10:15 - 12:00 Forelesning • A2	10:15 - 12:00 Forelesning • R1			
11					
12					
13					
14			14:15 - 16:00 4-Laboratorieøvelse • Smia 1 LA	14:15 - 16:00 8-Laboratorieøvelse • Lise 2 LAer	14:15 - 16:00 3-Laboratorieøvelse • Lise 1 LA
15					14:15 - 16:00 5-Laboratorieøvelse • Lise 1 LA
16					
17					
18					
19					

Innleveringstidspunkt

Alle kan gå på alle øvingstimer – men hvis alle venter til siste liten får ingen hjelp

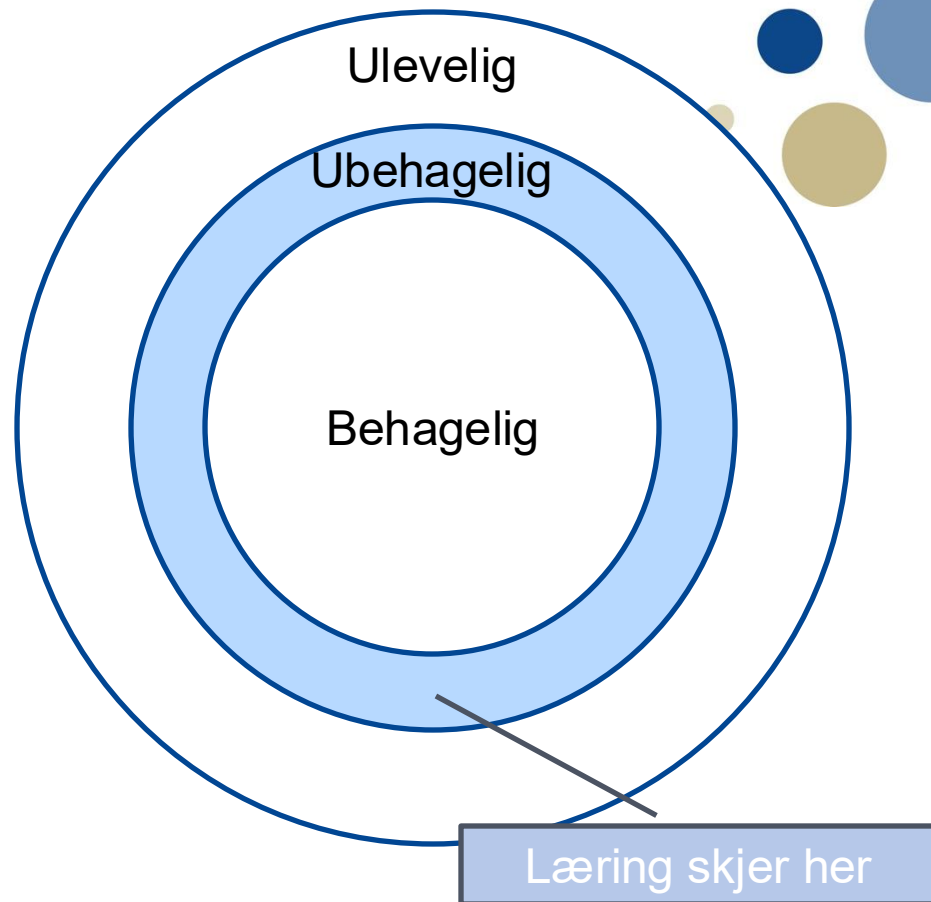
Skalerbarhet

- Hvorfor unngå epost?
 - Forumet er alle-til-alle
 - Samme spørsmål → et svar
 - Hele fagstaben kan svare
- Dere er ~800 studenter:
 - Anta at 10% sender en epost til meg i en gitt uke
 - Anta at jeg bruker 5 minutter på å svare på hver epost
 - $5 * 800 * 0.1 = 400$ minutter
= 6 timer og 40 minutter

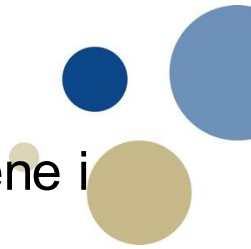


Studieteknikk

- Hvis du har lyst til å lære deg noe nytt må du øve
 - **Volum:** Du må bruke nok tid
 - **Intensitet:** Du må gjøre passe utfordrende oppgaver
- ... men hva skal du øve på?
 - Vi har valgt de læringsaktivitetene som vi **tror** vil gi flest mulig best mulig læringsutbytte



Pedagogikk



- Forelesningene vil fokusere på å forklare de viktigste momentene i hvert tema
 - Du vil neppe gjøre det veldig bra på eksamen hvis du bare går (ser) på forelesning
- Beste praksis for hvert tema (tror jeg):
 1. Skumles pensum som skal dekket (se semesterplan)
 2. Gå på forelesningen
 3. Les pensum igjen. Hvis noe er uklart, gå på sal og spør en læringsassistent eller spør på forumet.
- Mange synes boka er vanskelig å lese
 - ... og det er nok fordi det er det første møtet med en solid teknisk fagbok
 - Tips: Ikke prøv å forstå alt med en gang (se over)

Kvalitetssikring

- Hvordan kan vi vite at læringsaktivitetene gir det ønskede læringsutbyttet?
- Vi kan se på hva dere får til på eksamen og øvinger
 - ... men der er det mange variabler vi ikke kan kontrollere
- Vi kan spørre dere
 - En spørreundersøkelse etter semesteret
 - To spørreundersøkelser underveis (for å måle utvikling over tid)
 - Referansegruppe

Referansegruppe

- Interessert? Kom ned til meg i pausen



Forventet læringsutbytte

Kunnskaper:

- **K1:** Studenten skal kjenne til **datamaskiners konstruksjon og virkemåte**, inkludert hvordan man oppnår høy effektivitet.
- **K2:** Studenten skal forstå **grensesnittet mellom programvare og maskinvare**.
- **K3:** Studenten skal forstå hvordan man **bygger enkle og effektive prosessorer**, inkludert enkeltsykel, flersykel, og samlebåndsarkitekturer.
- **K4:** Studenten skal forstå prinsippene bak hvordan man bygger **effektive minnesystemer**, inkludert hurtigbuffer og virtuelt minne.
- **K5:** Studenten skal forstå hvordan **abstraksjon og struktur** benyttes for å oppnå **høy effektivitet** og til å **håndtere kompleksitet** i datamaskiner.

Ferdigheter:

- **F1:** Studenten skal være i stand til å formulere enkle programmer i **assemblykode**.
- **F2:** Studenten skal være i stand til å **lese blokkdiagrammer**.
- **F3:** Studenten skal kunne relatere **blokkdiagrammer** på ulike abstraksjonsnivå til hverandre.

Generell kompetanse:

- **G1:** Studenten skal forstå den generelle virkemåten til en datamaskin og kunne anvende denne kunnskapen i prosjekter på alle abstraksjonsnivå.

Forventet læringsutbytte (2)

- Denne beskrivelsen er for overordnet til å forstå hva som skal spesifikt skal læres
 - ... og vi har derfor laget spesifikke læringsutbyttebeskrivelser for hvert tema som er samlet i et dokument på Blackboard
- Dette dokumentet gir også eksempler på oppgaver i læreboka og gamle eksamensoppgaver for hvert tema
 - Merk: Det er en del feil i fasiten til læreboka, så vi er i ferd med å lage vår eget løsningsforslag for de anbefalte oppgavene
- Jeg kommer til å oppdatere dokumentet etter hvert som jeg ser hva jeg får dekket i hver forelesning



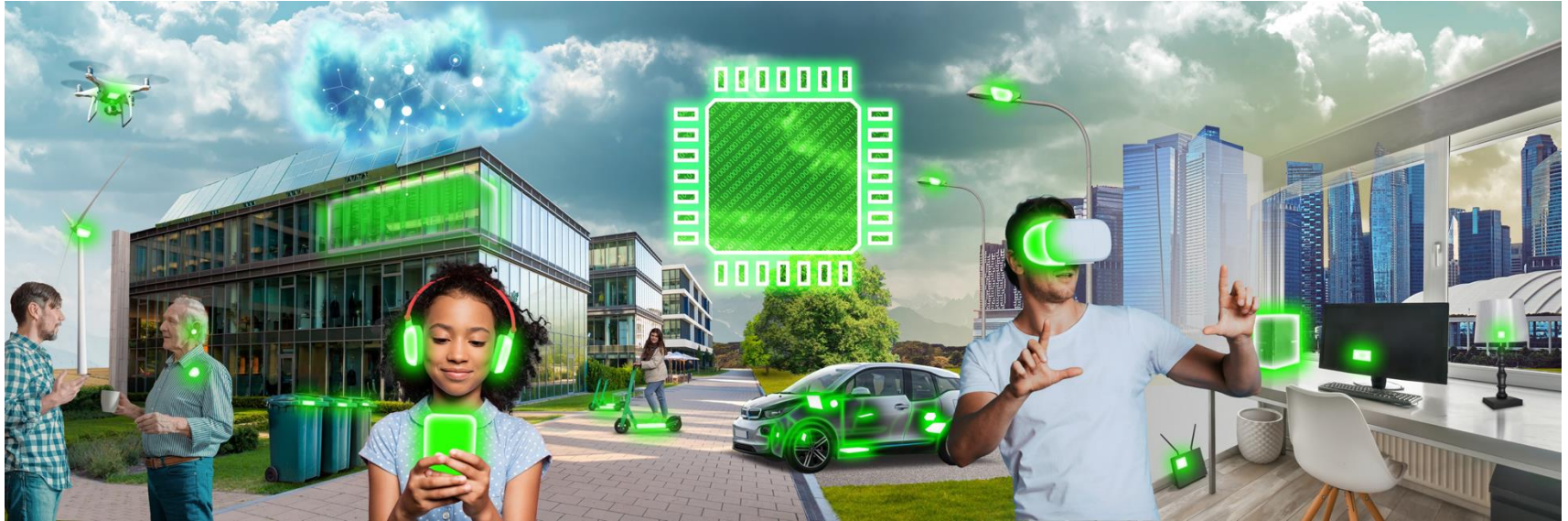
Tema 1.1 (Kapittel 1.1 og 1.2)

DATAMASKINTYPER OG DE 7 STORE IDEENE

Læringsutbytte T1.1

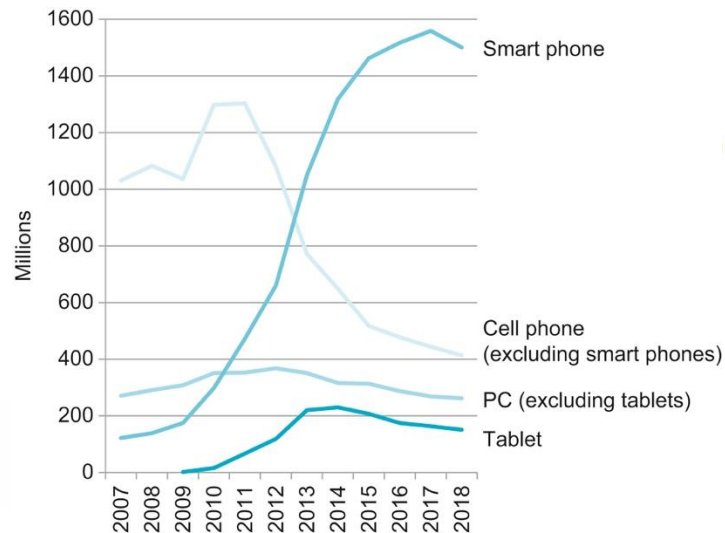
- Studenten kjenner til de 7 **datamaskintypene** og deres viktigste egenskaper
- Studenten kjenner til de 7 **store ideene** i datamaskinarkitektur
- Studenten kan beskrive **størrelser** med rett prefiks (for eksempel GB og GiB)

Datamaskintyper



Datamaskintyper

- Ultralaveffekt systemer
- Innvevde systemer
- Mobiltelefoner
- Bærbare datamaskiner
- Stasjonære datamaskiner
- Servere
- Datasentere og superdatamaskiner



Quiz!

- Det fundamentale ytelsesmålet er kjøretiden til ekte applikasjoner
 - Kjøretid (execution time): Tiden det tar å kjøre en applikasjon
 - Gjennomstrømning (throughput): Mengde arbeid gjort per tidsenhet
- Ytelse, energiforbruk, og effektforbruk henger sammen
 - Hvis du vil ha høyere ytelse, må du forvente at effektforbruket øker
 - Effektforbruket er raten datamaskinen forbruker energi
- Hva er effektforbruket for ulike datamaskintyper?

Størrelsen på datamaskiner varierer voldsomt

- Effektforbruket til ulike datamaskiner varierer med 12 størrelsesordener
- Forskjellene i ytelse er nesten like store
 - ~1.2 EFlop/s for Frontier
 - ~60 Miop/s for et vanlig innvevd system

Decimal term	Abbreviation	Value	Binary term	Abbreviation	Value	% Larger
kilobyte	KB	10^3	kibibyte	KiB	2^{10}	2%
megabyte	MB	10^6	mebibyte	MiB	2^{20}	5%
gigabyte	GB	10^9	gibibyte	GiB	2^{30}	7%
terabyte	TB	10^{12}	tebibyte	TiB	2^{40}	10%
petabyte	PB	10^{15}	pebibyte	PiB	2^{50}	13%
exabyte	EB	10^{18}	exbibyte	EiB	2^{60}	15%
zettabyte	ZB	10^{21}	zebibyte	ZiB	2^{70}	18%
yottabyte	YB	10^{24}	yobibyte	YiB	2^{80}	21%
ronnabyte	RB	10^{27}	robibyte	RiB	2^{90}	24%
queccabyte	QB	10^{30}	quebibyte	QiB	2^{100}	27%

De 7 store ideene

Abstraksjon

Gjør det vanlige tilfellet raskt

Ytelse gjennom parallellitet

Ytelse gjennom bruk av samlebånd

Ytelse gjennom prediksjon

Lag et hierarki av minneenheter

Pålitelighet gjennom redundans



Tema 1.2 (Kapittel 1.3 til 1.5)

UNDER OVERFLATEN

Læringsutbytte T1.2

- Studenten kjenner rollen til **applikasjonsprogramvare** og **systemprogramvare**, herunder operativsystemet og kompilatoren
- Studenten kan beskrive de fem **hovedkomponentene** i en datamaskin
- Studenten kan forklare **prinsippet om lagrede program**
- Studenten kan gjengi **produksjonsprosessen** for integrerte kretser

Hva skjer under programmet ditt?

- Mellom applikasjonen og maskinvaren finner vi systemprogramvare
- Operativsystemet
 - Håndterer periferienheter (I/O)
 - Allokere lagring og minne
 - Beskytter ulike applikasjoner fra hverandre
- Kompilatoren
 - Oversetter kode fra et høynivåspråk til maskinkode

High-level
language
program
(in C)

```
swap(size_t v[], size_t k)
{
    size_t temp;
    temp = v[k];
    v[k] = v[k+1];
    v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for RISC-V)

```
swap:
    slli x6, x11, 3
    add x6, x10, x6
    lw x5, 0(x6)
    lw x7, 4(x6)
    sw x7, 0(x6)
    sw x5, 4(x6)
    jalr x0, 0(x1)
```

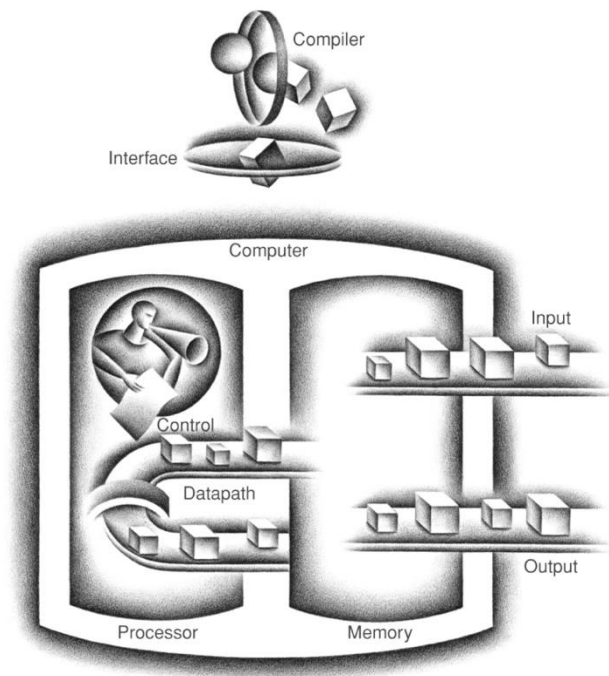
Assembler

Binary machine
language
program
(for RISC-V)

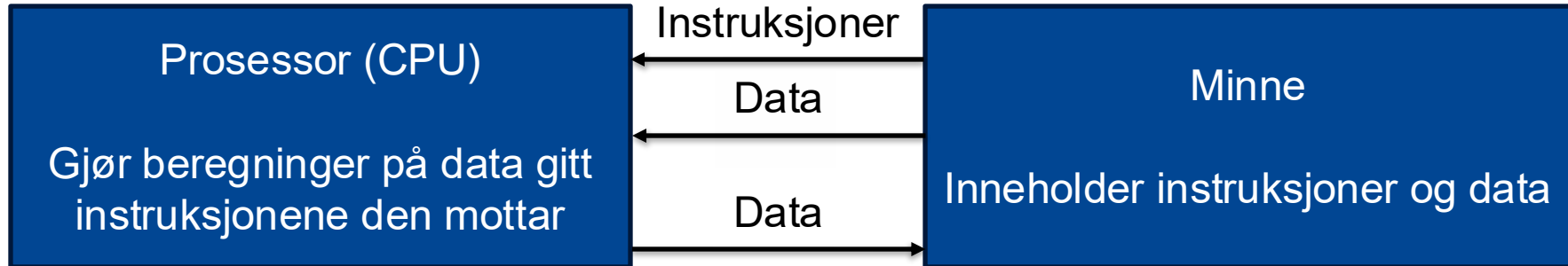
```
00000000001101011001001100010011
00000000011001010000001100110011
00000000000000110011001010000011
00000000100000110011001110000011
00000000011100110011000000100011
00000000010100110011010000100011
000000000000000100000001100111
```

Det store bildet

- En datamaskin består av fem komponenter:
 - Inndata
 - Utdata
 - Minne
 - Kontrollenhet
 - Datasti

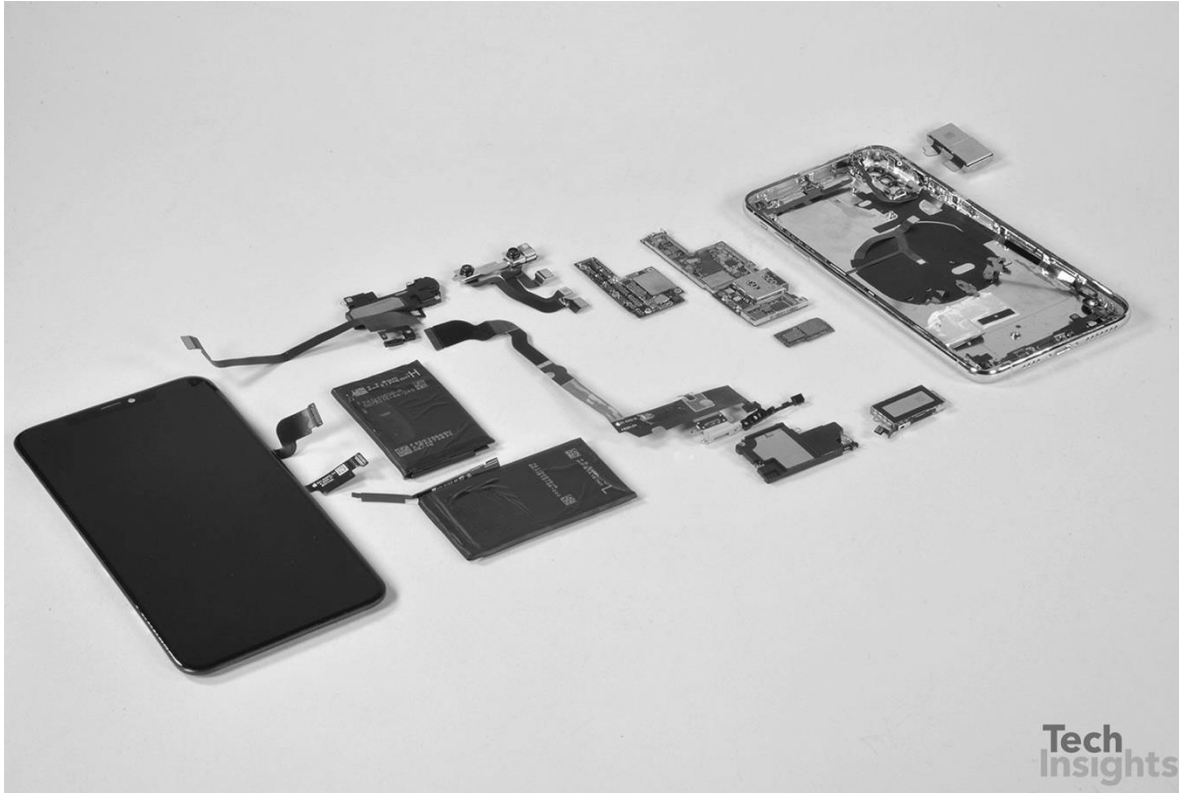


Prinsippet om lagrede program (stored program concept)

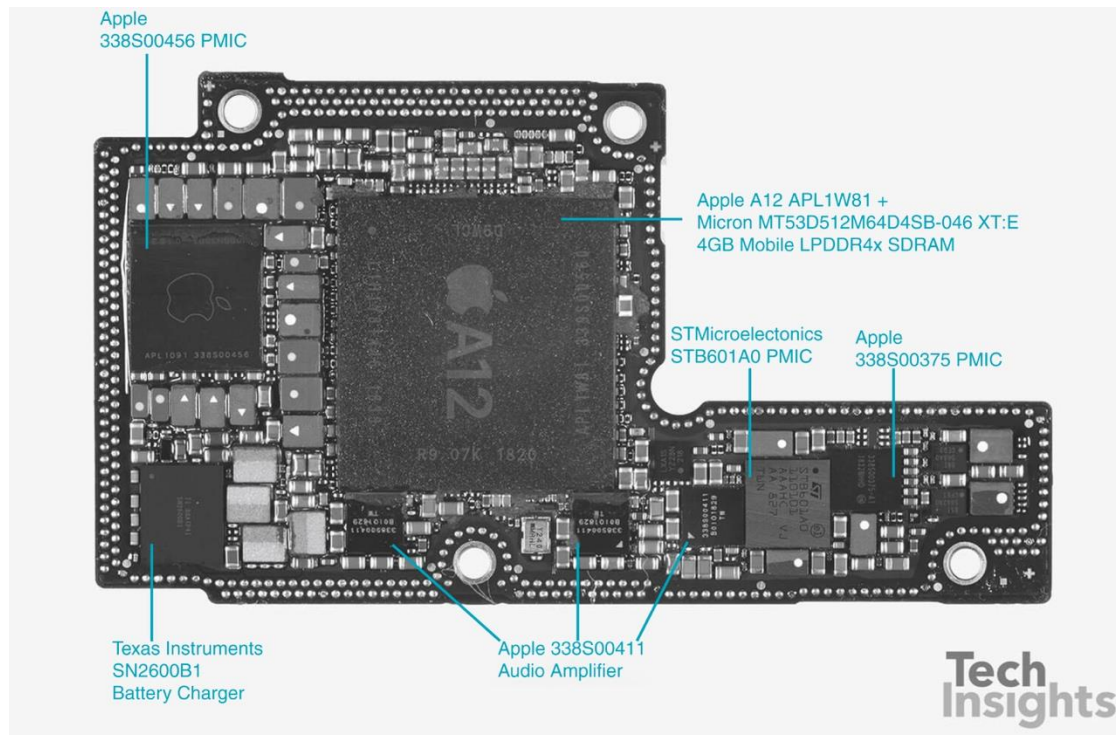


Inndata/Utdata (I/O)
Kommuniserer med omverdenen (skjerm, tastatur, nettverk, etc.)

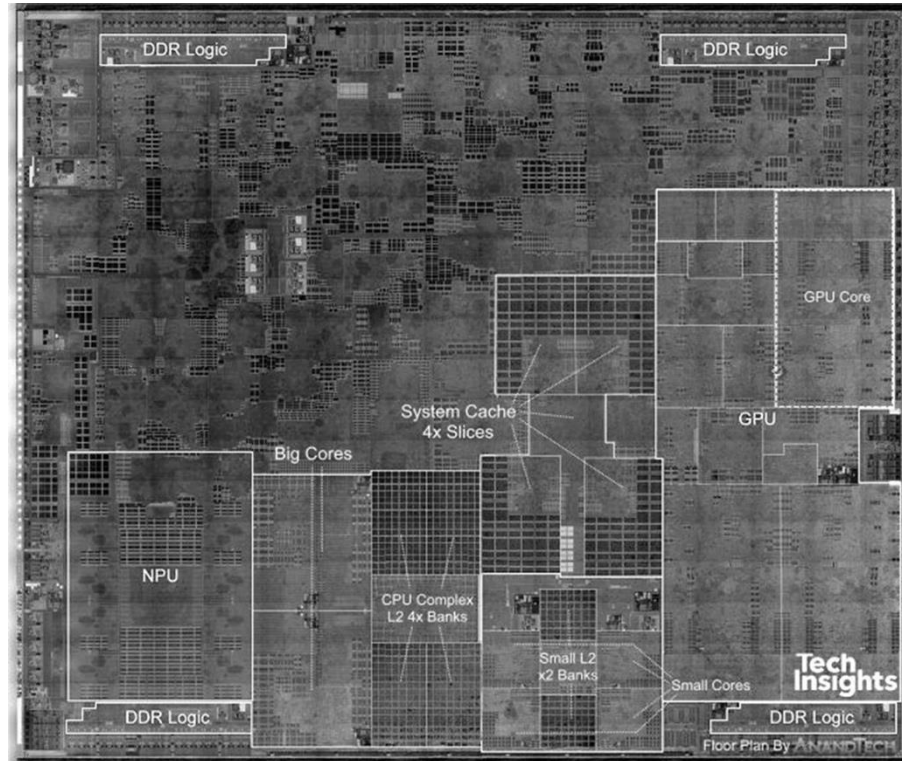
Inni en iPhone XS Max



Hovedkortet

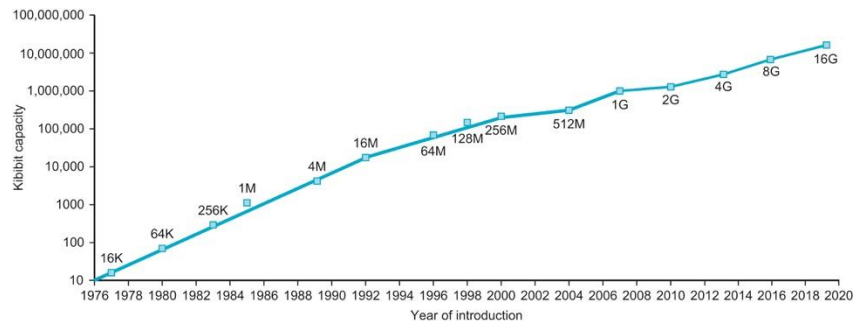


Inni A12 brikken



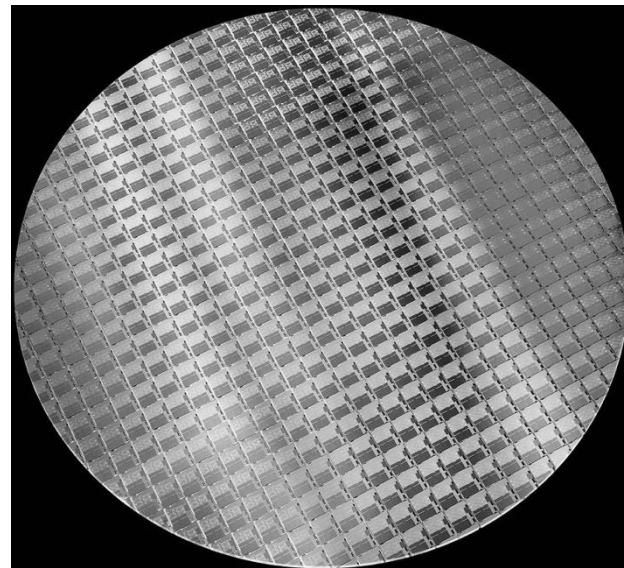
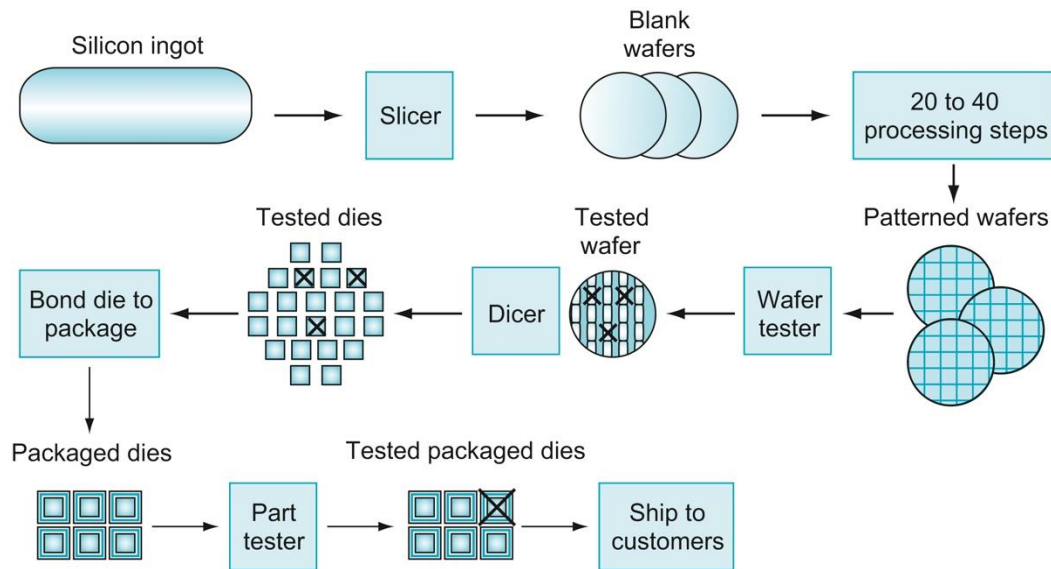
Integrerte kretser

- Transistor
 - En strømstyrt bryter
 - Å bytte fra av til på (eller motsatt) tar tid og bruker energi (dynamisk energiforbruk)
- En integrert krets er en brikke med mange transistorer
- Stort sett laget med silisium fordi man gjennom å kombinere det med ulike materialer kan lage ledere, isolatorer, og strømstyrte brytere



Minnekapasitet (DRAM) over tid

Produksjon av integrerte kretser





Tema 1.3 (Kapittel 1.6 og 1.7)

YTELSE

Læringsutbytte T1.3

- Studenten kjenner til de viktigste **ytelsesmetrikkene** i datamaskinarkitektur
- Studenten forklare forskjellen på **kjøretid** og **båndbredde** og velge den mest hensiktsmessige for gitte arkitekturoppgaver
- Studenten kan forklare «**The Iron Law**» og kan bruke den til å forutsi hvordan endringer i arkitekturen påvirker kjøretid
- Studenten kjenner til hvordan spenning og klokkefrekvens påvirker **effektforbruk** og **strømforbruk**
- Studenten vet hva en **testprogramsamling** er og hvorfor de brukes

Hvilket fly er best?



Airplane	Passenger capacity	Cruising range (miles)	Cruising speed (m.p.h.)	Passenger throughput (passengers × m.p.h.)
Boeing 737	240	3000	564	135,360
BAC/Sud Concorde	132	4000	1350	178,200
Boeing 777-200LR	301	9395	554	166,761
Airbus A380-800	853	8477	587	500,711

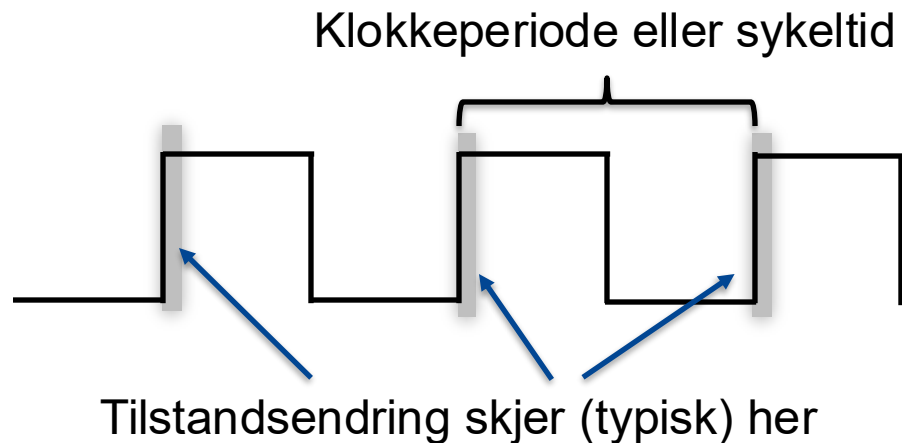
Quiz om ytelse

- Kjøretid (execution time)
 - Tiden det tar fra du starter et program til det er ferdig
 - Kan deles inn i tid brukt på applikasjonen (user time) og i operativsystemet (system time)
 - Lavere tall er bedre
- Gjennomstrømning (throughput)
 - Mengden arbeid gjort per tidsenhet
 - Høyere tall er bedre
- Ytelse er invers kjøretid
 - Høyere tall er bedre

$$\text{Performance}_X = \frac{1}{\text{Execution time}_X}$$

Klokke

- Datamaskiner er (stort sett) synkrone digitale systemer
- Det betyr at oppførselen til systemet kontrolleres av et klokkesignal
- Nå kan vi utlede «The Iron Law»



$$\text{Klokkefrekvens} = 1/\text{klokkeperiode}$$

Utlede «the Iron Law»

- «... the only complete and reliable measure of computer performance is time.»

«The Iron Law»

- «... the only complete and reliable measure of computer performance is time.»

$$\frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

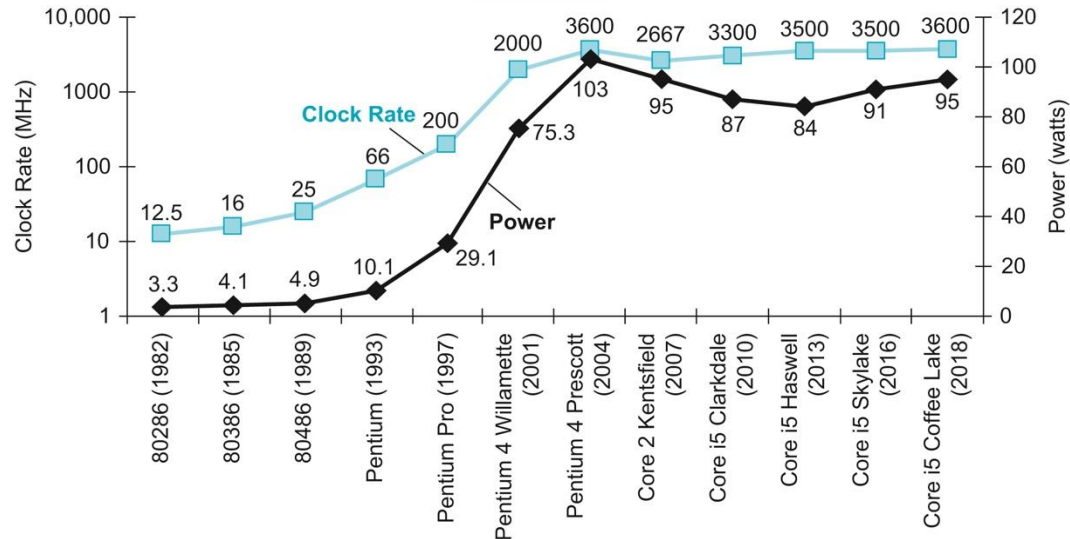
Sammenlikne ytelse (Quiz)



(Dynamisk) energiforbruk og effektforbruk

$$\text{Energy} \propto 1/2 \times \text{Capacitive load} \times \text{Voltage}^2$$

$$\text{Power} \propto 1/2 \times \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency switched}$$



Testprogrammer («Benchmarks»)

- Det fundamentale ytelsesmålet er kjøretiden til ekte programmer
- Hvilke programmer skal vi bruke?
 - Samlingen av testprogram bør være representativ for den typiske arbeidslasten til systemet («workload»)
- Hvis vi ønsker å gjøre det vanlige tilfellet raskt, må vi vite hva det vanlige tilfellet er
 - ... og vi må kunne karakterisere ytelsen til et system med ett enkelt tall

Testprogramsamlinger («Benchmark suites»)

- Det finnes en drøss testprogramsamlinger
 - SPEC CPU, TPC, MLPerf, Rodinia, ...
 - Ofte laget av et konsortium
 - Hver samling fokuserer på en type arbeidslast
- Det er flere måter å regne gjennomsnitt på og datamaskinarkitekter elsker å krangle om hvilket gjennomsnitt som er «rett»*

SPEC CPU2017 (speed) på en 1.8 GHz Intel Xeon E5-2650L

Description	Name	Instruction Count x 10 ⁹	CPI	Clock cycle time (seconds x 10 ⁻⁹)	Execution Time (seconds)	Reference Time (seconds)	SPECratio
Perl interpreter	perlbench	2684	0.42	0.556	627	1774	2.83
GNU C compiler	gcc	2322	0.67	0.556	863	3976	4.61
Route planning	mcf	1786	1.22	0.556	1215	4721	3.89
Discrete Event simulation - computer network	omnetpp	1107	0.82	0.556	507	1630	3.21
XML to HTML conversion via XSLT	xalancbmk	1314	0.75	0.556	549	1417	2.58
Video compression	x264	4488	0.32	0.556	813	1763	2.17
Artificial Intelligence: alpha-beta tree search (Chess)	deepsjeng	2216	0.57	0.556	698	1432	2.05
Artificial Intelligence: Monte Carlo tree search (Go)	leela	2236	0.79	0.556	987	1703	1.73
Artificial Intelligence: recursive solution generator (Sudoku)	exchange2	6683	0.46	0.556	1718	2939	1.71
General data compression	xz	8533	1.32	0.556	6290	6182	0.98
Geometric mean	–	–	–	–	–	–	2.36

*Eeckhout, «R.I.P. Geomean Speedup Use Equal-Work (Or Equal-Time) Harmonic Mean Speedup Instead», CAL, 2024, <https://ieeexplore.ieee.org/abstract/document/10419888>

Matrisemultiplikasjon (igjen)

Eksempel: Matrisemultiplikasjon

$$C = AB$$

```
for i in range(n):  
    for j in range(n):  
        for k in range(n):  
            C[i][j] +=  
                A[i][k] * B[k][j]
```

<i>n</i>	Naiv	Numpy	Speedup
128	0.223 s	0.004 s	~55X
1024	101.5 s	0.1 s	~1000X

